Mini proyecto LangChain

Explicador de conceptos

Guía para entender y explicar el proyecto, paso a paso

Python · LangChain · Claude (Anthropic)
Repositorio: github.com/juaneliaslarasoto/lang-chain

1. ¿Qué hace el proyecto?

Es un programa pequeño en Python. Le das un **tema** (por ejemplo fotosíntesis) y hace dos cosas:

1. Le pide a una inteligencia artificial (Claude) que explique el tema con palabras sencillas.

2. Con esa explicación, le pide que invente **una pregunta de repaso**.

Ejemplo de uso en la terminal:

```
$ python main.py fotosíntesis

Explicación:
La fotosíntesis es el proceso con el que las plantas fabrican su
propia comida usando la luz del sol, agua y dióxido de carbono...

Pregunta de repaso:
¿Qué tres cosas necesitan las plantas para hacer la fotosíntesis?
```

Explicación en una frase: «Es un programa que usa LangChain para conectar una plantilla de pregunta, un modelo de inteligencia artificial y un lector de respuestas en una "cadena", y luego usa el resultado de una cadena como entrada de otra.»

2. Conceptos básicos

¿Qué es un LLM?

Un **LLM** (Large Language Model, modelo de lenguaje grande) es una inteligencia artificial entrenada con muchísimo texto, capaz de entender y escribir lenguaje humano. Ejemplos: Claude, GPT, Gemini. En este proyecto usamos **Claude**, de la empresa Anthropic.

¿Qué es LangChain?

LangChain es una librería (un conjunto de herramientas ya hechas) para Python que facilita crear programas que usan LLMs. En lugar de programar todo desde cero, te da **piezas reutilizables** que se conectan entre sí, como bloques de LEGO.

Su nombre lo dice: **Lang** (de language, lenguaje) + **Chain** (cadena). La idea es **encadenar** pasos.

¿Por qué usar LangChain y no llamar a la IA directamente?

Ventaja	Explicación
Orden	Cada pieza hace una sola cosa; el código queda limpio y fácil de leer.
Reutilizar	Una plantilla de prompt se puede usar muchas veces con datos distintos.
Cambiar de IA fácilmente	Si cambias Claude por GPT, solo cambias una línea; el resto del código sigue igual.

Ventaja	Explicación
Encadenar pasos	Puedes conectar varias llamadas a la IA, donde la salida de una es la entrada de la siguiente.

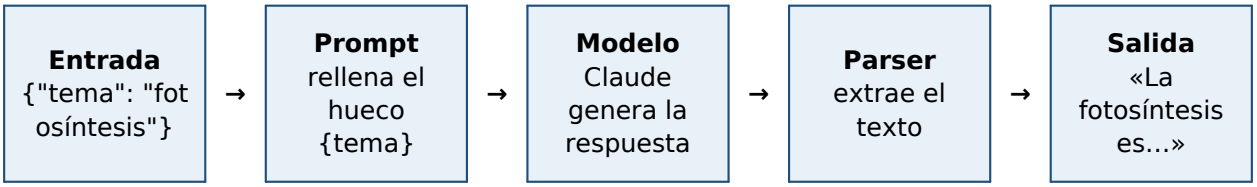
3. Las 3 piezas que usa el proyecto

Pieza	Nombre en el código	¿Qué hace?	Comparación
Plantilla de prompt	`ChatPromptTemplate`	Un mensaje con huecos ({tema}) que se rellenan después.	Un formulario con espacios en blanco.
Modelo (LLM)	`ChatAnthropic`	Envía el mensaje a Claude por internet y recibe la respuesta.	El cerebro que piensa la respuesta.
Parser de salida	`StrOutputParser`	Saca solo el texto de la respuesta (la IA devuelve un objeto con más datos).	Abrir el sobre y quedarse con la carta.

La cadena (chain)

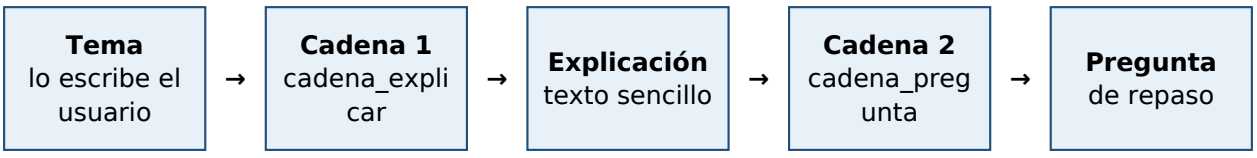
Las piezas se unen con el símbolo `|` (se llama pipe, tubería). Lo que sale de una pieza entra en la siguiente:

```
cadena_explicar = prompt_explicar | modelo | parser
```



Las dos cadenas del proyecto

El proyecto tiene **dos cadenas**, y la segunda usa el resultado de la primera:



Para ejecutar una cadena se usa `.invoke()`, pasándole un diccionario con los valores de los huecos.

4. El código explicado paso a paso

Todo el programa está en el archivo `main.py`. Veámoslo por partes.

Paso 1 – Importar las librerías

```
from dotenv import load_dotenv
from langchain_anthropic import ChatAnthropic
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
```

Traemos las herramientas que vamos a usar: `load_dotenv` para leer la clave secreta, y las tres piezas de LangChain.

Paso 2 – Cargar la API key

```
load_dotenv()

if not os.getenv("ANTHROPIC_API_KEY"):
    sys.exit("Falta ANTHROPIC_API_KEY. ...")
```

La **API key** es como una contraseña que permite usar Claude. Se guarda en un archivo `.env` que **nunca se sube a GitHub** (por eso está en `.gitignore`). Si falta, el programa avisa y se detiene.

Paso 3 – Crear el modelo

```
modelo = ChatAnthropic(
    model=os.getenv("MODELO", "claude-opus-5-5"),
    max_tokens=4000,
)
```

`model` indica qué versión de Claude usar (se puede cambiar en el `.env`). `max_tokens` es el límite de largo de la respuesta (un token es más o menos un trozo de palabra).

Paso 4 – Las plantillas de prompt

```
prompt_explicar = ChatPromptTemplate.from_messages([
    ("system", "Eres un profesor paciente. Explicas en español, ..."),
    ("human",  "Explícame '{tema}' en máximo 3 frases, como si tuviera 12 años."),
])
```

Hay dos tipos de mensaje:

- **system**: instrucciones de comportamiento para la IA («eres un profesor paciente»).
- **human**: lo que pregunta el usuario. `{tema}` es un hueco que se rellena al ejecutar.

La segunda plantilla (`prompt_pregunta`) funciona igual, pero con el hueco `{explicacion}`.

Paso 5 – El parser y las cadenas

```
parser = StrOutputParser()

cadena_explicar = prompt_explicar | modelo | parser
cadena_pregunta = prompt_pregunta | modelo | parser
```

Aquí está el corazón de LangChain: unir las piezas con `|`. Fíjate en que el mismo modelo y el mismo parser se reutilizan en las dos cadenas.

Paso 6 – La función principal

```
def main():
    tema = " ".join(sys.argv[1:]) or input("¿Qué tema quieres entender? ")

    explicacion = cadena_explicar.invoke({"tema": tema})
    print("Explicación:\n" + explicacion)

    pregunta = cadena_pregunta.invoke({"explicacion": explicacion})
    print("Pregunta de repaso:\n" + pregunta)
```

1. Lee el tema: si lo escribiste al ejecutar (`python main.py agujeros negros`) lo usa; si no, te lo pregunta.
2. Ejecuta la primera cadena con el tema → obtiene la explicación.
3. Ejecuta la segunda cadena con la explicación → obtiene la pregunta.
4. Imprime ambos resultados.

5. Archivos del proyecto

Archivo	Para qué sirve
`main.py`	Todo el código del programa (unas 50 líneas comentadas).
`requirements.txt`	Lista de librerías que hay que instalar.
`.env.example`	Plantilla para crear el archivo .env con tu API key.
`.gitignore`	Le dice a Git qué archivos NO subir (como .env, para proteger la clave).
`README.md`	Documentación que se ve en la página de GitHub.

6. Cómo ejecutarlo

```
# 1. Crear y activar un entorno virtual (opcional, recomendado)
python -m venv venv
source venv/bin/activate          # En Windows: venv\Scripts\activate

# 2. Instalar las librerías
pip install -r requirements.txt

# 3. Crear el archivo .env y poner la clave de console.anthropic.com
cp .env.example .env              # En Windows: copy .env.example .env

# 4. Ejecutar
python main.py agujeros negros
```

Errores comunes

Error	Solución
Falta ANTHROPIC_API_KEY	No creaste el archivo .env o la clave está vacía.
ModuleNotFoundError: langchain_anthropic	Falta instalar: pip install -r requirements.txt (¿activaste el venv?).
401 authentication_error	La API key es incorrecta.

7. Glosario

Término	Significado
LLM	Modelo de lenguaje grande: IA que entiende y genera texto (Claude, GPT…).
Prompt	El mensaje o instrucción que se le da a la IA.
Plantilla (template)	Un prompt con huecos {así} que se rellenan con datos.
Chain (cadena)	Varias piezas conectadas: la salida de una es la entrada de la siguiente.

Término	Significado
Parser	Pieza que transforma la respuesta de la IA a un formato útil (aquí, texto).
invoke()	Método para ejecutar una cadena con unos datos de entrada.
API key	Clave secreta para poder usar el servicio de la IA.
.env	Archivo donde se guardan datos secretos; no se sube a GitHub.
Token	Trozo de texto (más o menos una sílaba o palabra corta) con el que trabaja la IA.
Entorno virtual (venv)	Carpeta aislada con las librerías del proyecto, para no mezclarlas con otras.

8. Preguntas que te pueden hacer (y cómo responder)

Pregunta	Respuesta
¿Qué es LangChain?	Una librería de Python para construir aplicaciones con modelos de lenguaje, uniendo piezas en cadenas.
¿Qué significa el símbolo | ?	Conecta piezas: la salida de la izquierda pasa como entrada a la derecha.
¿Para qué sirve el parser?	La IA devuelve un objeto con texto y datos extra; el parser se queda solo con el texto.
¿Qué diferencia hay entre system y human?	system define cómo se comporta la IA; human es la pregunta del usuario.
¿Por qué hay dos cadenas?	Para mostrar cómo encadenar pasos: la explicación de la primera alimenta a la segunda.
¿Por qué no se sube el .env?	Porque tiene la API key; si se publica, otros podrían usarla y gastar tu dinero.
¿Se puede usar otra IA?	Sí: se cambia ChatAnthropic por, por ejemplo, ChatOpenAI; el resto del código no cambia.

Ideas para ampliarlo: añadir un hueco {nivel} (niño / universitario / experto); usar .stream() para ver la respuesta letra a letra; cambiar el «profesor» para que hable como un pirata.